React DevTools

Version: 19.3 **Website:** react.dev

The Problem

React DevTools allows developers to understand the UI of their programs in an incredibly accessible way. The normal Chrome Inspect shows only the final HTML output, not the components, props, state, or hooks that produced it. Without a dedicated tool, debugging means re-running the app repeatedly, with no live view of what each component holds at any moment. React DevTools solves this problem by exposing the full component tree, the props and state of every node, and a Profiler that records exactly which components re-rendered, why, and how long each render took — turning invisible React internals into an inspectable, real-time graph.

Why React DevTools

- It measures instead of guessing. The flame graph shows real render times from your own interaction.
- It needs no code changes. Libraries like why-did-you-render must be wired into the app; React DevTools does not.
- It understands React Compiler. Components the compiler optimized show a Memo ✨ badge, so you can see what is and isn't auto-memoized.

Setup

1. Install the Chrome Extension
2. Open your project on Google Chrome
3. Right click to Inspect the page or use Ctrl+Shift+I
4. Navigate to the Profiler and Components tabs

When to Use It ✅/ When Not To ❌

✅ Good fit	❌ Not a good fit
Locating components in the source code	The slowness is the network or the database (use the Network tab or server logs)
Typing, scrolling, or clicking feels laggy	The app isn't built with React
You want to know which component re-renders, and why	You only have a production build and need timings (the Profiler needs a dev or profiling build)
You need to check props, state, or hooks live	The slow code isn't React. I.E., in Actual, budget math and queries run in loot-core in a background worker, which DevTools can't see

Cost: Completely free. No account required.

Tutorial Video

https://www.youtube.com/watch?v=tHrCCIb5Nto